

RationaleAnchor: Preserving Architectural Intent Across Human and AI Transitions

Mihai Rosca

Independent Researcher, BRIDGRAI Ecosystem

Braila, Romania, EU

ORCID: 0009-0001-1422-6209

Date: September 2026 | Status: Preprint — pending peer review | License: CC BY 4.0

Abstract

Knowledge continuity engineering (KCE) addresses how knowledge persists across transitions of AI models, human personnel, and regulatory frameworks. However, KCE preserves *what* was built, not *why* it was built. When the original architect becomes unavailable, systems retain code, hashes, and documentation but lose the rationale for design decisions, non-goals, and invariants that must be maintained. This paper introduces **RationaleAnchor**, a specification and reference implementation for preserving architectural intent as a first-class, verifiable artifact. RationaleAnchor defines a YAML schema capturing purpose, canonical goals, non-goals, invariants, dependencies, evidence, status, and human authority for every significant artifact. It provides automated validation of structural integrity, bidirectional dependency consistency, invariant verifiability, and succession plan executability. We present the reference implementation (Python 3.10+, zero mandatory dependencies) with a CLI for validation, dependency graphing, governance compliance checking, conflict detection, and succession planning. We validate RationaleAnchor on three artifacts from the BRIDGRAI ecosystem (KCE, CAMP, SIGNA), demonstrating bidirectional dependency verification and governance compliance. RationaleAnchor extends KCE with Layer 0 (Intent), resolving the *bus factor of intent*: when the original architect is unavailable, structured intent records preserve the rationale for design decisions.

Keywords: architectural intent, design rationale, knowledge continuity, software architecture, bus factor, intent preservation, succession planning

1. Introduction

1.1 The Bus Factor of Intent

Software systems accumulate knowledge over time: code, tests, documentation, deployment scripts. Knowledge continuity engineering (KCE) [1] addresses how this knowledge persists across transitions—when AI models are deprecated, when human personnel leave, when regulatory frameworks change.

However, KCE preserves *what* was built, not *why* it was built. Provenance systems answer “where did this artifact come from?” Version control answers “what changed and when?” Documentation answers “how does this work?” None answer the question that matters most when the original architect is unavailable:

Why does this artifact exist, and what must remain invariant?

This is the **bus factor of intent**. When the architect who made the design decisions becomes unavailable—through departure, death, or organizational restructuring—the system retains its code

and documentation but loses the rationale for its existence. Successors can see what was built but cannot distinguish canonical decisions from experimental ones, cannot know which invariants must be maintained, and cannot understand why certain approaches were explicitly rejected.

1.2 The Gap in Existing Approaches

Architecture Decision Records (ADRs) [2] capture individual decisions but do not provide structural validation, dependency tracking, or succession planning. ADRs are prose documents; they cannot be automatically verified for consistency.

Design rationale literature [3, 4] establishes the importance of preserving reasoning but focuses on capture methods, not on executable verification or continuity mechanisms.

Provenance systems (W3C PROV [5], OpenTimestamps [6]) track artifact lineage but do not preserve the intent behind creation.

Knowledge continuity engineering [1] addresses knowledge persistence but treats intent as implicit within knowledge artifacts, not as a separate, verifiable layer.

None of these approaches provide: (1) a structured schema for intent that can be automatically validated, (2) bidirectional dependency consistency checking, (3) invariant verifiability enforcement, (4) succession plan executability validation, or (5) conflict detection across artifacts with contradictory invariants.

1.3 RationaleAnchor

RationaleAnchor addresses this gap by treating architectural intent as a first-class artifact with its own schema, validation rules, and continuity mechanisms. RationaleAnchor provides:

- **Structured intent records:** YAML schema capturing purpose, canonical goals, non-goals, invariants, dependencies, evidence, status, and human authority
- **Automated validation:** structural integrity, bidirectional dependency consistency, invariant verifiability, status coherence
- **Governance compliance:** verification that invariants are verifiable and succession plans are executable
- **Conflict detection:** identification of contradictory invariants across artifacts
- **Reference implementation:** Python CLI with zero mandatory dependencies

RationaleAnchor extends KCE with Layer 0 (Intent), completing the stack: Intent (why) → Creation (what) → Validation (quality) → Persistence (storage) → Governance (authority) → Continuity (transitions).

1.4 Paper Structure

Section 2 reviews related work. Section 3 presents design principles. Section 4 defines the formal specification. Section 5 describes the reference implementation. Section 6 presents a case study on the BRIDGRAI ecosystem. Section 7 discusses limitations and future work. Section 8 concludes.

2. Related Work

2.1 Design Rationale

Design rationale research establishes that preserving reasoning is critical for maintenance and evolution. Tang [3] surveys rationale capture methods and their impact on design quality. Burge and Drummond [4] analyze rationale representation formats and their trade-offs.

However, design rationale literature focuses on *capture*—how to record reasoning during design—not on *continuity*—how to ensure reasoning remains accessible, verifiable, and actionable across organizational transitions. RationaleAnchor addresses continuity through structured schemas, automated validation, and succession planning.

2.2 Architecture Decision Records

ADRs [2] are lightweight documents capturing individual architectural decisions: context, decision, consequences. ADRs are widely adopted in industry but have limitations:

- **No structural validation:** ADRs are prose; consistency cannot be automatically verified
- **No dependency tracking:** ADRs do not model relationships between decisions
- **No succession planning:** ADRs do not address what happens when the decision-maker leaves
- **No invariant enforcement:** ADRs record decisions but do not verify that invariants are maintained

RationaleAnchor extends the ADR concept with executable validation, dependency graphs, and continuity mechanisms.

2.3 Provenance Systems

W3C PROV [5] provides a data model for provenance: entities, activities, agents, and their relationships. OpenTimestamps [6] provides blockchain-based existence proofs.

Provenance systems answer “where did this come from?” but not “why does this exist?” A PROV record can state that artifact X was created by agent Y at time T, but cannot express that X exists to solve problem P, must maintain invariants I, and must never do N.

RationaleAnchor complements provenance by preserving intent. The two are orthogonal: provenance tracks lineage, RationaleAnchor tracks purpose.

2.4 Knowledge Continuity Engineering

KCE [1] proposes a five-layer architecture for knowledge continuity: Creation, Validation, Persistence, Governance, Continuity. KCE addresses how knowledge survives transitions but treats intent as implicit within knowledge artifacts.

RationaleAnchor extends KCE with Layer 0 (Intent), making intent explicit, structured, and verifiable. This resolves the bus factor of intent identified in KCE: when the original architect is unavailable, RationaleAnchor records preserve the rationale for design decisions.

3. Design Principles

3.1 Intent ≠ Knowledge

Knowledge answers “what” and “how.” Intent answers “why” and “why not.” A codebase can be fully documented (knowledge complete) while its intent is lost (no one knows why certain approaches were rejected or which invariants are canonical).

RationaleAnchor treats intent as a separate artifact type with its own schema, not as metadata embedded within knowledge artifacts.

3.2 Intent Must Be Verifiable

An intent record that cannot be verified is documentation, not engineering. RationaleAnchor requires that invariants be verifiable through one of four methods:

- **regex:** pattern matching against artifact content
- **file_exists:** check for required files
- **test_passes:** execute test suite to validate invariant
- **manual_review:** human review with documented procedure

Invariants that are not verifiable are flagged as governance violations.

3.3 Dependencies Must Be Bidirectional

If artifact A influences artifact B, then B must list A in its `influenced_by` field. Unidirectional dependencies create orphaned references and make impact analysis impossible. RationaleAnchor validates bidirectional consistency across all artifacts in a directory.

3.4 Succession Must Be Executable

A succession plan that states “Heritage Agent manages succession” is a template, not a plan. Executable succession plans must include:

- **Successor ID:** person or system instance
- **Trigger condition:** dormancy period, explicit handoff, etc.
- **Verification method:** how to confirm successor is operational

RationaleAnchor flags template succession plans as governance violations.

3.5 Status Must Be Coherent

An artifact cannot be simultaneously canonical and deprecated, or deprecated and experimental. Status combinations are validated to prevent contradictory states.

4. Formal Specification

4.1 Schema

RationaleAnchor records use a YAML schema with seven sections: **artifact** (`id`, `version`, `type`), **purpose** (`why_this_exists`, `problem_solved`), **intent** (`canonical_goal`, `non_goals`, `invariants`, `success_criteria`), **dependencies** (`requires`, `influences`, `influenced_by`), **evidence** (`tests`, `publications`, `blockchain_anchor`, `reviews`), **status** (`canonical`, `experimental`, `deprecated`, `last_reviewed`), and **human_authority** (`decision_owner`, `succession_plan`, `intent_documentation`).

Supported artifact types: `paper`, `code`, `story`, `protocol`, `product`, `framework`, `tool`.

4.2 Validation Rules

Structural validation:

- All required fields present; type constraints satisfied
- `artifact.type` is a supported value
- `status.last_reviewed` is a valid ISO 8601 date

Self-reference validation: artifact cannot appear in its own `influences`, `influenced_by`, or `requires`.

Status coherence:

- `canonical` and `deprecated` cannot both be true
- `deprecated` and `experimental` cannot both be true

Bidirectional dependency consistency: if A lists B in `influences`, then B must list A in `influenced_by`.

Conflict detection: identical invariants across different artifacts generate conflict warnings.

4.3 Governance Policy

RationaleAnchor enforces seven governance rules:

Rule	Description
1	Artifacts exceeding 100 hours MUST have a RationaleAnchor record
2	Safety-critical artifacts MUST have one regardless of effort
3	Intent MUST be reviewed quarterly by the decision owner
4	Deprecation requires a migration path or archival rationale
5	Influence relationships MUST be bidirectional
6	Invariants MUST be verifiable
7	Succession plans MUST be executable

5. Reference Implementation

5.1 Architecture

The reference implementation is written in Python 3.10+ with zero mandatory dependencies. PyYAML is optional; the implementation includes a minimal YAML parser for the repository's simple schema.

Modules:

- `schema.py`: Schema definition and structural validation
- `validator.py`: Record validation, directory validation, invariant checking, conflict detection
- `cli.py`: Command-line interface with 7 commands

5.2 CLI Commands

Command	Description
<code>validate <path></code>	Validate single file or directory
<code>graph <path></code>	Display dependency graph
<code>check-drift <path></code>	Detect structural and invariant drift
<code>diff <old> <new></code>	Compare two intent records
<code>intent-governance <path></code>	Check governance compliance
<code>conflict-check <path></code>	Detect contradictory invariants
<code>succession-plan <path></code>	Validate succession plan executability

5.3 Validation Pipeline

1. Load YAML: Parse file with PyYAML (if available) or minimal parser
2. Structural validation: Check schema conformance
3. Self-reference validation: Check for circular dependencies
4. Status coherence: Check for contradictory status flags
5. Bidirectional consistency: Check dependency graph symmetry (directory mode)

6. Invariant verifiability: Check that invariants are verifiable (governance mode)
7. Conflict detection: Check for contradictory invariants across artifacts (conflict mode)

5.4 Test Suite

The reference implementation includes 7 automated tests: KCE record validation, CAMP record validation, SIGNA record validation, bidirectional dependency consistency, canonical/deprecated conflict detection, invariant verifiability checking, and conflict detection. All tests pass on Python 3.10+.

6. Case Study: BRIDGRAI Ecosystem

6.1 Artifacts

We applied RationaleAnchor to three artifacts from the BRIDGRAI ecosystem:

KCE (Knowledge Continuity Engineering) [1]: Framework for knowledge persistence across AI model rotation, human turnover, and regulatory change. Published: DOI 10.5281/zenodo.22667873.

CAMP (Cognitive Arbitration Mediation Protocol): Vendor-neutral protocol for trust verification across multiple AI providers. Published: DOI 10.5281/zenodo.22682610.

SIGNA (Semantic Intent Guarantee and Notary Architecture): Certification framework for AI output trustworthiness across six validation dimensions. Version 1.2 (9 patches applied). Published: DOI 10.5281/zenodo.22682891.

6.2 Dependency Graph

```
KCE --influences--> CAMP
KCE --influences--> SIGNA
CAMP --influences--> SIGNA
SIGNA --influences--> CAMP
```

Bidirectional consistency validated: all influence relationships have corresponding `influenced_by` entries. CAMP and SIGNA have mutual influence (bidirectional link enforced by the validator).

6.3 Governance Compliance

Invariants:

- KCE: 3 invariants (five problems addressed, seven principles enforced, human anchor required)
- CAMP: 3 invariants (structural independence, independent verifiability, cryptographic anchoring)
- SIGNA: 3 invariants (per-pillar floor $\tau=0.40$, six pillars independent, scoring reproducible)

Succession plans: All three artifacts specify “Patrick Rosca (transgenerational continuity)” as successor with the decision owner being Mihai Rosca. The governance check validates these as executable (named person, not a template).

Status: All three artifacts are canonical, not experimental, not deprecated.

6.4 Conflict Detection

No conflicts detected: no identical invariants across artifacts. Each artifact's invariants are domain-specific and non-contradictory.

7. Discussion

7.1 Limitations

Semantic drift detection is structural, not semantic. The `check-drift` command validates structural integrity but does not detect semantic drift (e.g., implementation no longer aligns with declared intent). Semantic drift detection requires an external evaluator.

Conflict detection is syntactic. Identical invariants across artifacts are flagged as potential conflicts, but semantic contradiction detection (e.g., “must be fast” vs “must be thorough”) requires natural language understanding.

Minimal YAML parser is limited. The fallback parser supports the repository’s simple schema but not arbitrary YAML. PyYAML is recommended for production use.

7.2 Future Work

- **Intent versioning:** Track intent changes over time with semantic versioning and migration rationale
- **Semantic drift detection:** Integrate external evaluators (LLM-based or test-based) to detect when implementation no longer aligns with declared intent
- **Heritage Agent integration:** Execute succession plans automatically when decision owners become unavailable
- **Cross-model review integration:** Document cross-model analysis as evidence of disagreement/coverage, not independent certification

7.3 Integration with KCE

Layer	Concern	Source
L0: Intent	Why	RationaleAnchor
L1: Creation	What	KCE
L2: Validation	Quality	KCE
L3: Persistence	Storage	KCE
L4: Governance	Authority	KCE
L5: Continuity	Transitions	KCE

This completes the KCE stack, resolving the bus factor of intent.

8. Conclusion

Knowledge continuity engineering preserves *what* was built. RationaleAnchor preserves *why* it was built.

When the original architect becomes unavailable, systems retain code, hashes, and documentation but lose the rationale for design decisions, non-goals, and invariants. RationaleAnchor addresses this gap by treating architectural intent as a first-class artifact with structured schema, automated validation, and continuity mechanisms.

The reference implementation provides validation, dependency graphing, governance compliance checking, conflict detection, and succession planning. Applied to three artifacts from the BRIDGRAI ecosystem, RationaleAnchor demonstrates bidirectional dependency verification and governance compliance.

RationaleAnchor extends KCE with Layer 0 (Intent), completing the stack and resolving the bus factor of intent. When the architect is unavailable, structured intent records preserve the rationale for design decisions.

References

- [1] Rosca, M. (2026). Knowledge Continuity Engineering: A Framework for Persistent Knowledge Across Human and AI Generations. Zenodo. <https://doi.org/10.5281/zenodo.22667873>
 - [2] Nygard, M. (2011). Documenting Architecture Decisions. Cognitect Blog.
 - [3] Tang, A. (2007). Towards a unified model of design rationale. *Design Studies*, 28(4), 363-385.
 - [4] Burge, J. E., & Drummond, T. (2018). Design rationale: past, present, and future. *AI EDAM*, 32(3), 257-261.
 - [5] W3C. (2013). PROV-DM: The PROV Data Model. W3C Recommendation.
 - [6] OpenTimestamps. (2016). Blockchain-based timestamping. <https://opentimestamps.org>
-

AI Contribution Statement

Research phase: Prior art and naming verification conducted using multiple AI models (Claude, Qwen, ChatGPT, Gemini) with cross-model disagreement analysis. No single model was treated as authoritative.

Drafting phase: Paper structure and content drafted with assistance from Claude (Anthropic). All content reviewed, corrected, and adopted by the human author.

Naming: “RationaleAnchor” name verified as unique through cross-model search and human verification.

The author bears full responsibility for all claims.

Data Availability

All RationaleAnchor components referenced in this paper are publicly available:

- Reference implementation: 7 CLI commands, zero mandatory dependencies
- Test suites: 7 automated tests, all passing
- Examples: KCE, CAMP, SIGNA intent records
- Related work: KCE framework (DOI 10.5281/zenodo.22667873)